Chapitre 8 : Récursivité : Complexité

ITC - 2025/2026

Félix POTIÉ

Sommaire

Sommaire 2
1. Complexité des algorithmes récursifs 4
1.1 Exemple : 1 ou 2 appels récursifs 5
1.2 Relation de récurrence 10
1.3 Exercices 15
2. Cas des algorithmes utilisant la méthode divisé pour régner 21
2.1 Exemple : recherche dichotomique 22
2.2 Master theorem 26
2.3 Exercice 1 : max 28
2.4 Exercice 2 : cas des slices 31
2.5 Autres exercices 35

1. Complexité des algorithmes récursifs “basique”

1. Complexité des algorithmes récursifs

Jusqu'à présent, nous avons étudié la complexité des boucles. Ces complexités permettent de déterminer la complexité de la plupart des algorithmes itératifs.

Pour les algorithmes récursifs, l'étude de la complexité est souvent plus complexe.

1.1 Exemple : 1 ou 2 appels récursifs

Soit la suite (u_n) suivante, qui calcule une approximation de $\sqrt{3}$:

- $u_0 = 2$
- $\forall n \in \mathbb{N}^*, u_n = \frac{1}{2}\left(u_{n-1} + \frac{3}{u_{n-1}}\right)$

```
def u(n): # à ne pas faire
    if n == 0:
        return 2
    else:
        return 0.5 * (u(n-1) + 3/(u(n-1)))
```

Ici dans certains cas on appelle 2 fois la même fonction avec les mêmes paramètres. On pourrait croire que ce ne change pas grand chose mais en fait si !

1.1 Exemple : 1 ou 2 appels récursifs - 2

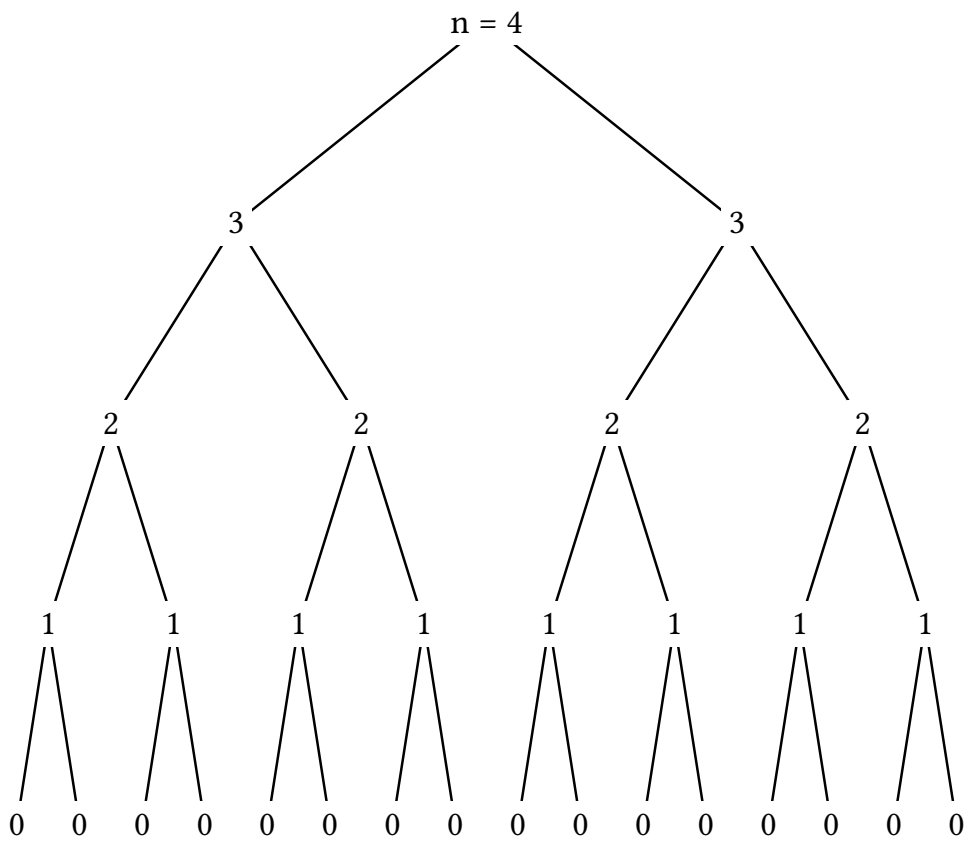
Il vaut mieux écrire celà où l'on a qu'un appel à la fonction récursive.

```
def u(n):
    if n == 0:
        return 2
    else:
        x = u(n-1)
        return 0.5 * (x + 3/x)
```

Comparons la complexité des 2 algorithmes.

1.1 Exemple : 1 ou 2 appels récursifs - 3

Avec la fonction aux 2 appels récursif on aurait :



1.1 Exemple : 1 ou 2 appels récursifs - 4

Alors qu'avec la version où l'on aurait qu'un appel on aurait :

1.1 Exemple : 1 ou 2 appels récursifs - 5

On a une intuition qu'avec la première version on a un algorithme en $O(2^n)$ contre un $O(n)$ pour la deuxième.

1.2 Relation de récurrence

Pour analyser la complexité d'une fonction récursive, on peut utiliser des **relations de récurrence**.

La complexité d'un algorithme récursif peut être décomposée en deux termes :

1. la complexité due aux calculs :
   - **séparant les données** pour réaliser les appels,
   - **reconstruisant le résultat** après les appels.

   Donc la complexité de la fonction sans les appels récursifs.

2. la complexité due aux **appels récursifs**

1.2 Relation de récurrence - 2

Revenons sur la cas de notre suite.

1. - Coût de la **séparation** des données : calcul de `n-1` en $O(1)$.
   - Coût de la **reconstruction** du résultat : $O(1)$, on fait que des calculs arithmétiques de bases.
     Le coût total de ces opérations est donc $O(1)$.
2. Coût des **appels récursifs** : on fait 2 appel sur `n-1` (1 appel sur `n-1` dans le cas de la deuxième version)

On a alors la relation de récurrence :

$$C(n) = O(1) + 2C(n-1) \text{ , pour la v1}$$
$$C(n) = O(1) + C(n-1) \text{ , pour la v2}$$

où C(n) est la compléxité de la fonction en fonction de n.

1.2 Relation de récurrence - 3

On résoud la relation de récurrence :

- Version avec 1 seul appel récursif :

On sait que $C(0) = O(1)$ car c'est notre cas de base.

$$
\begin{aligned}
C(n) &= O(1) + C(n-1) \\
&= 2 \cdot O(1) + C(n-2) \\
&\text{... après } k \text{ dépliages ...} \\
&= k \cdot O(1) + C(n-k) \\
&\qquad \text{on pose } k = n \\
&= n \cdot O(1) + C(0) \\
&= O(n) + O(1) \\
&= O(n)
\end{aligned}
$$

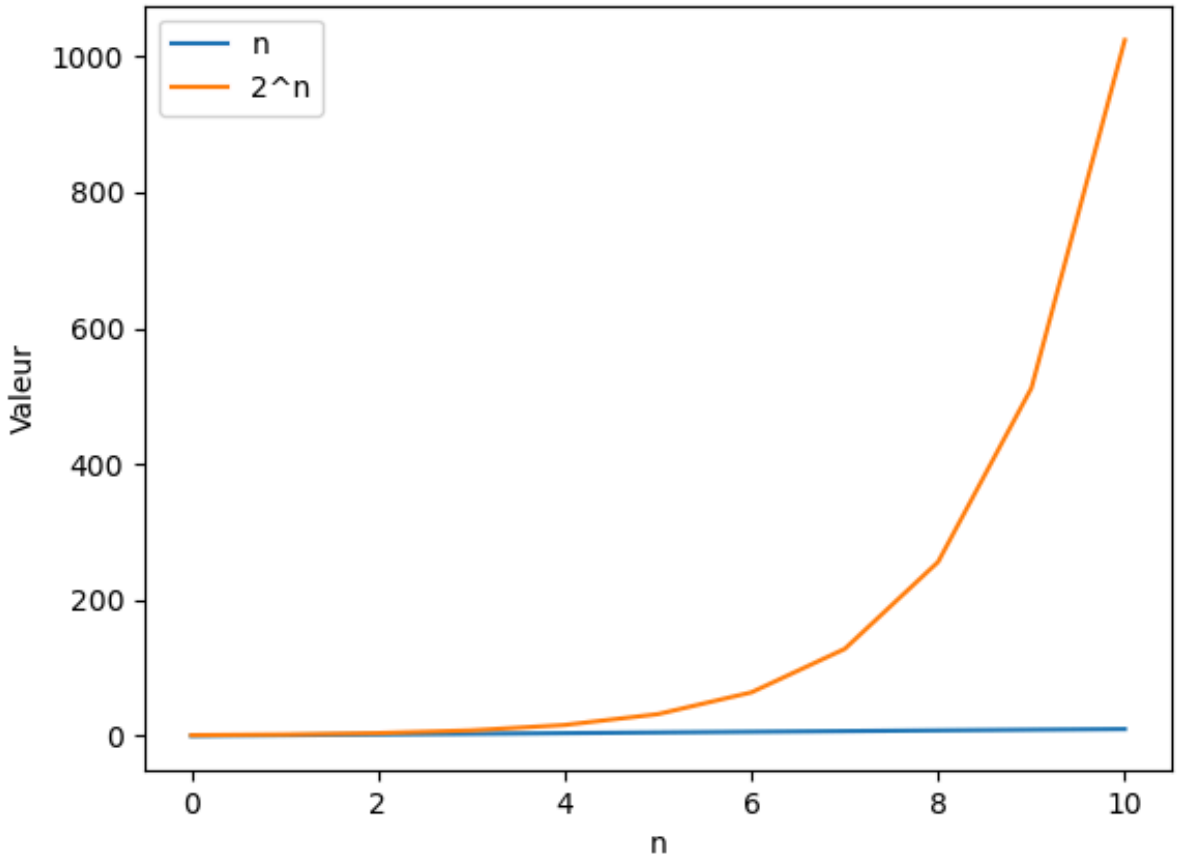
1.2 Relation de récurrence - 4

- Version avec 2 appels récursifs :

$$
\begin{aligned}
C(n) &= O(1) + 2 \cdot C(n-1) \\
&= O(1) + 2 \cdot (O(1) + 2 \cdot C(n-2)) \\
&= O(1) \cdot (1+2) + 4 \cdot C(n-2) \\
&\ldots \text{ après } k \text{ dépliages} \\
&= O(1) \cdot \left(1 + 2 + 2^2 + \cdots + 2^{k-1}\right) + 2^k \cdot C(n-k) \\
&\qquad \text{on pose } k = n \\
&= O(1) \cdot \left(1 + 2 + \cdots + 2^{n-1}\right) + 2^n \cdot C(0) \\
&= O(1) \cdot (2^n - 1) + O(1) \cdot 2^n \\
&= O(2^n)
\end{aligned}
$$

1.2 Relation de récurrence - 5

Il est donc très important de ne pas appelé 2 fois la fonction avec le même paramètres.



1.3 Exercices

```
def f(n):
    if n == 0:
        return 1
    else :
        return f(n-1) * f(n-1) * f(n-1)
```

Question Trouver la relation de récurrence, en déduire la complexité de cette fonction.

1.3 Exercices - 2

1. • Coût de la **séparation** des données : calcul de `n-1` en $O(1)$.
   • Coût de la **reconstruction** du résultat : $O(1)$

2. Coût des **appels récursifs** : on fait 3 appel sur `n-1`

$$
\begin{aligned}
C(n) &= O(1) + 3 \cdot C(n) \\
&= O(1) + 3 \cdot (O(1) + 3 \cdot C(n-2)) \\
&\text{... après } k \text{ dépliages} \\
&= O(1) \cdot \left(1 + 3 + 3^2 + \cdots + 3^{k-1}\right) + 3^k \cdot C(n-k) \\
&\qquad \text{on pose } k = n \\
&= O(1) \cdot (3^n - 1) + O(1) \cdot 3^n \\
&= O(3^n)
\end{aligned}
$$

1.3 Exercices - 3

```
def f(n):
    if n == 0:
        return 1
    else :
        x = 0
        y = f(n-1)
        for i in range(n):
            x = x + 1
        return x + y
```

Question Trouver la relation de récurrence, en déduire la complexité de cette fonction.
Attention ici la complexité du traitement n'est pas $O(1)$.

1.3 Exercices - 4

1. - Coût de la **séparation** des données : calcul de `n-1` en $O(1)$.
   - Coût de la **reconstruction** du résultat : $O(n)$ on a une boucle de taille n.

   Donc $O(n)$ pour le traitement.

2. Coût des **appels récursifs** : on fait 1 appel sur `n-1`

1.3 Exercices - 5

$$
\begin{aligned}
C(n) &= O(n) + C(n-1) \\
&= O(n) + O(n-1) + C(n-2)) \\
&\text{... après } k \text{ dépliages} \\
&= O(n) + O(n-1) + ... + O(n-k+1) + C(n-k) \\
&\qquad \text{on pose } k = n \\
&= O(n) + O(n-1) + ... + O(2) + O(1) + O(1) \\
&= O\left(\frac{n(n+1)}{2}\right) \\
&= O(n^2)
\end{aligned}
$$

2. Cas des algorithmes utilisant la méthode divisé pour régner

2. Cas des algorithmes utilisant la méthode divisé pour régner

Pour les algorithmes utilisant la méthode diviser pour régner et la récursivité, nous allons utiliser un théorème général très connu : **le master theorem**.

2.1 Exemple : recherche dichotomique

```python
def rech_dicho(liste, x, debut, fin):
    if debut > fin:
        return None  # x non trouvée

    milieu = (debut + fin) // 2

    if liste[milieu] == x:
        return milieu
    elif liste[milieu] < x:
        return rech_dicho(liste, x, milieu+1, fin)
    else:
        return rech_dicho(liste, x, debut, milieu-1)

def recherche_dichotomique(liste, x):
    return rech_dicho(liste, x, 0, len(liste) - 1)
```

2.1 Exemple : recherche dichotomique - 2

1. - Coût de la **séparation** des données : calcul de `milieu` en $O(1)$.
   - Coût de la **reconstruction** du résultat : nul, on retourne directement le résultat.

   Le coût total de ces opérations est donc $O(1)$.

2. Coût des **appels récursifs** : les paramètres importants sont que le problème initial est divisé en 1 seul sous-problème de taille $\frac{n}{2}$. (On a un appel récursif et la |fin-debut| est divisé par 2).

2.1 Exemple : recherche dichotomique - 3

Diviser pour régner	**Rech. dichotomique**
Donnée de taille n	Liste de taille n
Traitement $O(n^k)$	$O(1)$, $k = 0$
Morceaux de taille $\frac{n}{b}$	$b = 2$
a nombre de sous-problèmes	$a = 1$

On a donc la relation de récurrence suivante :

$$C(n) = a \times C\left(\frac{n}{b}\right) + O\left(n^k\right)$$

Pour la recherche dichotomique on a donc :

$$C(n) = 1 \times C\left(\frac{n}{2}\right) + O(1)$$

2.1 Exemple : recherche dichotomique - 4

$$
\begin{aligned}
C(n) &= C\left(\frac{n}{2}\right) + O(1) \\
&= \left(C\left(\frac{n}{4}\right) + O(1)\right) + O(1) \\
&= \left(C\left(\frac{n}{8}\right) + O(1)\right) + 2 \times O(1) \\
&= \ldots \\
&= C\left(\frac{n}{2^k}\right) + k \times O(1) \\
\text{on pose } k &= \log_2(n) \text{ pour avoir :} \\
&= C(1) + \log_2(n) \times O(1) \\
&= O(\log(n))
\end{aligned}
$$

2.2 Master theorem

Au lieu de résoudre la relation de récurrence, nous pouvons utiliser le master theorem.

Définition 1: Master Theorem.

Soit $a \geq 1$, $b > 1$, $k \geq 0$ trois constantes, et $C(n)$ définie par récurrence par :

$$C(n) = a \times C\left(\frac{n}{b}\right) + O\left(n^k\right) \qquad \text{alors,}$$

- Si $a < b^k$ alors $C(n) = O\left(n^k\right)$
- Si $a = b^k$ alors $C(n) = O\left(n^k \cdot \log(n)\right)$
- Si $a > b^k$ alors $C(n) = O\left(n^{\log_b(a)}\right)$

2.2 Master theorem - 2

Exemple : sur la fonction de recherche dichotomique
$C(n) = 1 \times C(\frac{n}{2}) + O(1)$

$a = 1, b = 2, k = 0$ donc $b^k = 2^0 = 1$
On est alors dans le cas $a = b^k$, donc :
$C(n) = O\big(n^k \log(n)\big) = O\big(n^0 \log(n)\big) = O(\log(n))$

On retrouve bien le même résultat qu'en dépilant la formule de récurrence.

2.3 Exercice 1 : max

Calcul du maximum : on le calcule à la manière d'un tournoi en divisant la liste en 2 et en prenant le max des max des 2 sous-listes. Trouver la relation de récurrence et donc la complexité.

```
def splitmax_aux(l, debut, fin):
    if fin - debut == 1 :
        return l[debut]
    milieu = (debut + fin)//2
    m1 = splitmax_aux(l, debut, milieu)
    m2 = splitmax_aux(l, milieu, fin)
    return max(m1, m2)

def splitmax(l):
    return splitmax_aux(l, 0, len(l))
```

2.3 Exercice 1 : max - 2

On pose n la taille de la liste.
La complexité du traitement est :

- $O(1)$ pour le calcul de `milieu` et
- $O(1)$ pour la reconstruction du résultat (le max entre les 2 maxs).

On a alors $O(1)$ pour le traitement.

On découpe la liste en 2 et on appelle la fonction récursivement 2 fois sur ces listes.

On a donc la formule de récurrence :
$C(n) = 2 \times C(\frac{n}{2}) + O(1)$

2.3 Exercice 1 : max - 3

On a la formule de récurrence :
$C(n) = 2 \times C(\frac{n}{2}) + O(1)$

On applique donc le master theorem avec
$a = 2, b = 2$ et $k = 0$.

On a alors, $a > b^k$ donc $C(n) = O\left(n^{\log_2(2)}\right) = O(n)$

On retrouve donc la complexité **linéaire** qu'on avait avec l'algorithme itératif.

2.4 Exercice 2 : cas des slices

Si l'on avait implémenté le même algorithme à l'aide de slices :

```
def splitmax(l):
    if len(l) == 1 :
        return l[0]
    milieu = len(l)//2
    m1 = splitmax(l[:milieu])
    m2 = splitmax(l[milieu:])
    return max(m1, m2)
```

2.4 Exercice 2 : cas des slices - 2

On pose n la taille de la liste.
La complexité du traitement est :

- $O(2 * \frac{n}{2}) = O(n)$ pour la création des slices
- $O(1)$ pour la reconstruction du résultat (le max entre les 2 maxs).

On a alors $O(n + 1) = O(n)$ pour le traitement.

On découpe la liste en 2 et on appelle la fonction récursivement 2 fois sur ces listes.

On a donc la formule de récurrence :
$C(n) = 2 \times C(\frac{n}{2}) + O(n)$

2.4 Exercice 2 : cas des slices - 3

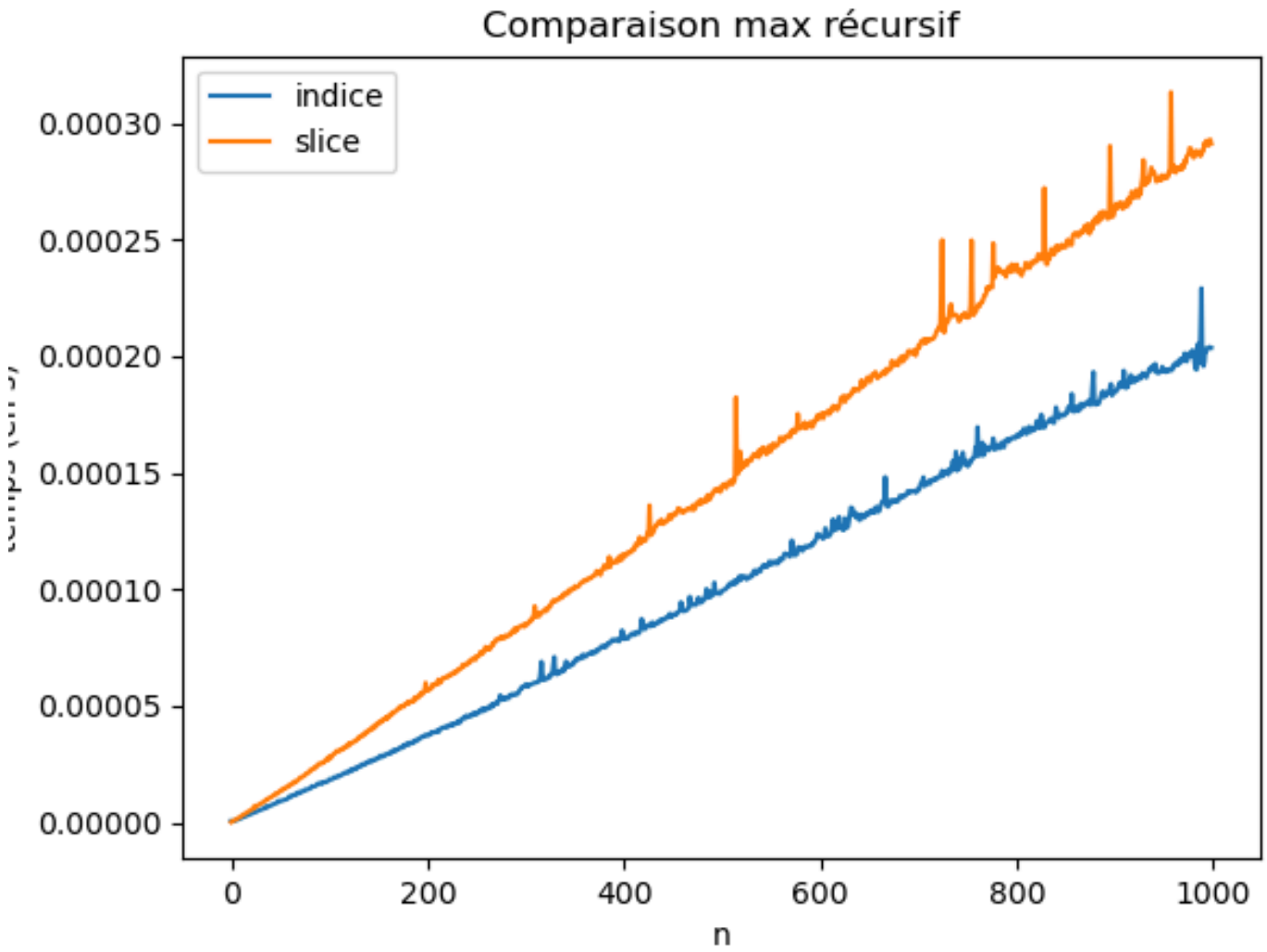
On a la formule de récurrence :
$C(n) = 2 \times C(\frac{n}{2}) + O(n)$

On applique donc le master theorem avec
$a = 2, b = 2$ et $k = 1$.

On a alors, $a = b^k$ donc
$C(n) = O(n^1 \log(n)) = O(n \log(n))$

On **perd** donc la complexité **linéaire**, c'est pourquoi on passe par des indices et pas par des slices.

2.4 Exercice 2 : cas des slices - 4



2.5 Autres exercices

```
def exo1(n):
    if n <= 1:
        return 1
    k = 0
    for i in range(n):
        for j in range(i):
            k = k+1
    k = k + exo1(int(n/3))
    k = k + exo1(int(n/3))
    k = k + exo1(int(n/3))
    return k
```

2.5 Autres exercices - 2

La complexité du traitement est :

- $O(n^2)$ à cause des boucles for imbriquées
- $O(1)$ pour la reconstruction du résultat, addition avec k

On a alors $O(n^2 + 1) = O(n^2)$ pour le traitement.

On découpe n en 3 et on appelle la fonction récursivement 3 fois.

On a donc la formule de récurrence :
$C(n) = 3 \times C\left(\frac{n}{3}\right) + O(n^2)$

On applique donc le master theorem avec
$a = 3, b = 3$ et $k = 2$.

On a alors, $a < b^k$ donc
$C(n) = O(n^2)$

2.5 Autres exercices - 3

```
def exo2(n):
    if n <= 1:
        return 1
    k = 0
    for i in range(25):
        k = k + exo2(int(n/5))
    for i in range(n):
        for j in range(n):
            k = k+1
    return k
```

2.5 Autres exercices - 4

La complexité du traitement est :

- $O(1)$ on divise juste par n par 5
- $O(n^2)$ pour la reconstruction avec les 2 boucles for imbriquées

On a alors $O(1 + n^2) = O(n^2)$ pour le traitement.

On découpe n en 5 et on appelle la fonction récursivement 25 fois.

On a donc la formule de récurrence :
$C(n) = 25 \times C\left(\frac{n}{5}\right) + O(n^2)$

On applique donc le masterm theorem avec
$a = 25, b = 5$ et $k = 2$.

On a alors, $a = b^k$ donc
$C(n) = O\left(n^2 \log(n)\right)$